

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University

Accredited by NAAC with "B++" grade



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PCCSL407 OPERATING SYSTEMS LAB

LAB MANUAL

PREPARED BY:

Ms. RAMYA RAJ K P

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

- M1: Seek to foster in every student a profound and practical understanding of knowledge, encouraging critical inquiry and intellectual curiosity.
- M2: Strive to ensure every student develops holistically, cultivating the skills, ethical character, and resilience necessary to thrive in diverse and challenging future environments.
- M3: Scale the commitment to service by inspiring students to apply their knowledge and skills to actively address community needs, promoting ethical leadership and social responsibility.

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge.

MISSION OF THE DEPARTMENT

- M1: To seek and strive for advancements in technological domains.
- M2: To create socially responsible engineers in the field of computer science.
- M3: To bridge the gap between science and mankind through meaningful innovation and service.

Program Educational Objectives (peos)

PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

- PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
- PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
- PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
- PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
- PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
- PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
- PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
- PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

OPERATING SYSTEMS LAB

<i>Program:</i> B. Tech	<i>Department:</i> Engineering
<i>Course:</i> OS Lab	<i>Semester:</i> S4 <i>Batch:</i> C <i>Credits:</i> 2
<i>Course Code:</i> PCCSL407	<i>Regulation:</i> 2024
<i>Course Area/Domain:</i> General Engineering	L: 0 T: 0 TW/SL: 3 P: 3
Corresponding Lab Course Code (If Any): Nil	<i>Lab Course Name:</i> Nil

COURSE PRE-REQUISITES:

<i>CODE</i>	<i>COURSE NAME</i>	<i>DESCRIPTION</i>	<i>SEM</i>
PCCST407	Operating Systems Lab	To familiarize various Linux commands related to Operating systems.	S4

Course Objectives:

1. To familiarize various Linux commands related to Operating systems.
2. To give practical experience for learners on implementing different functions of Operating systems such as process management, memory management, and disk management.

Course Outcomes (COs)

Expt. No.	Experiments
1	Familiarisation with basic Linux programming commands: ps, strace, gdb, strings, objdump, nm, file, od, xxd, time, fuser, top
2	Use /proc file system to gather basic information about your machine: a. Number of CPU cores b. Total memory and the fraction of free memory c. Number of processes currently running.

	d. Number of processes in the running and blocked states. e. Number of processes forked since the last bootup. How do you compare this value with the one in (c) above? f. The number of context switches performed since the last bootup for a Particular process.
3	Write a simple program to print the system time and execute it. Then use the /proc file system to determine how long this program (in the strict sense, the corresponding process) ran in user and kernel modes.
4	Create a new process using a fork system call. Print the parent and child process IDs. Use the pstree command to find the process tree for the child process starting from the initprocess.
5	Write a program to add two integers (received via the command line) and compile it to an executable named “ myadder ”. Now write another program that creates a new process using a fork system call. Make the child process add two integers by replacing its image with the “ myadder ” image using execvp system call.
6	Create a new process using a fork system call. The child process should print the string “ PCCSL407 ” and the parent process should print the string “ Operating Systems Lab ”. Use a wait system call to ensure that the output displayed is “ PCCSL407 Operating Systems Lab ”
7	Inter-process Communication (https://www.linuxdoc.org/LDP/lpg/node7.html) a. Using Pipe – Evaluate the expression. The first process evaluates. The second process evaluates and sends it to the first process which evaluates the final expression and displays it. b. Using Message Queue - The first process sends a string to the second process. The second process reverses the received string and sends it back to the first process. The first process compares the original string and the reversed string received from the second one and then prints whether the string is a palindrome or not. c. Using Shared Memory - The first process sends three strings to the second process. The second process concatenates them to a single string (with whitespace being inserted between the two individual strings) and sends it back to the first process. The first process prints the concatenated string in the flipped case, that is if the concatenated string is “Hello S4 Students”, the final output should be “hELLO s4 sTUDENTS”

8	Write a multithreaded program that calculates the mean, median, and standard deviation for a list of integers. This program should receive a series of integers on the command line and will then create three separate worker threads. The first thread will determine the mean value, the second will determine the median and the third will calculate the standard deviation of the integers. The variables representing the mean, median, and standard deviation values will be stored globally. The worker threads will set these values, and the parent thread will output the values once the workers have exited.
9	process mix, determining which algorithm results in the minimum average waiting time (over all processes).
10	Use semaphores to solve the readers-writers problem with writers being given priority over readers.
11	Obtain a (deadlock-free) process mix and simulate the banker's algorithm to determine a safe execution sequence.
12	Obtain a process mix and determine if the system is deadlocked.
13	Implement the deadlock-free semaphore-based solution for the dining philosopher's problem.
14	Simulate the address translation in the paging scheme as follows: The program receives three command line arguments in the order size of the virtual address space (in megabytes) page size (in kilobytes) a virtual address (in decimal notation) The output should be the physical address corresponding to the virtual address in <frame number, offset> format. You may assume that the page table is implemented as an array indexed by page numbers. (NB: If the page table has no index for the page number determined from the virtual address, you may just declare a page table miss!)
15	Simulate the FIFO, LRU, and optimal page-replacement algorithms as follows: First, generate a random page-reference string where page numbers range from 0 to 9. Apply the random page-reference string to each algorithm, and record the number of page faults incurred by each

	algorithm. Assume that demand paging is used. The length of the reference string and the number of page frames (varying from 1 to 7) are to be received as command line arguments.
16	Simulate the SSTF, LOOK, and CSCAN disk-scheduling algorithms as follows: Your program will service a disk with 5,000 cylinders numbered 0 to 4,999. The program will generate a random series of 10 cylinder requests and service them according to each of the algorithms listed earlier. The program will be passed the initial position of the disk head (as a parameter on the command line) and will report the total number of head movements required by each algorithm.

COURSE OUTCOMES:

<i>SL NO</i>	<i>DESCRIPTION</i>
CO1	Illustrate the use of various system calls in Operating Systems.
CO2	Implement process creation and compare the performance of various disk scheduling algorithms
CO3	Compare the performance of various CPU scheduling algorithms
CO4	Compare the performance of various disk scheduling algorithms
CO5	Understand the storage management in operating systems

CO - PO/PSO MAPPING:

CO Number	PO STRENGTH											PSO STRENGTH	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	3	3	3				3			3	2	2
CO2	3	3	3	3				3			3	3	3
CO3	3	3	3	3				3			3	3	2
CO4	3	3	3	3				3			3	3	2
CO5	3	3	3	3				3			3	3	2

GENERAL INSTRUCTIONS (for the lab)

Preparation:

- Read the theory, algorithm, and expected output for the experiment scheduled before the lab session. Preparation is key to successful implementation.
- Familiarize yourself with the basic functionality of the Linux/Unix environment you will be using (e.g., shell navigation, basic command utilities).
- Bring your Lab Manual/Record to every session to document procedures, code snippets, and results in real-time.
- For simulation exercises (e.g., Scheduling, Deadlock), prepare the required input values (process details, resource matrix, reference strings) beforehand.

Lab Setup & Tools:

- Ensure you are logged into the assigned Linux/Unix machine or virtual environment.
- Familiarize yourself with the command-line editor (e.g., nano, vim) or IDE (e.g., VS Code, Eclipse) available for C development.
- Confirm the availability and basic usage of the GNU Compiler Collection (gcc) for compiling your C/C++ programs.
- Create a new directory for each experiment (e.g., mkdir exp_04_fork) to keep your source code and executables organized.

Coding Standards & Implementation:

- Language: All system programming tasks (IPC, Synchronization, Process Management) must be implemented using C or C++, utilizing standard system calls.
- Header Comments: Start every C/C++ file with a comment block including your name, roll number, date, experiment number, and the program's purpose.
- System Calls: Use robust error handling (checking the return value against -1 and using perror()) immediately after every critical system call (e.g., fork(), exec(), shmget(), semop()).
- Compilation Flags: Use necessary flags, such as -o to name the executable and -pthread when compiling multithreaded programs.
Example: gcc myprogram.c -o myprog -pthread
- Code Clarity: Use clear, descriptive variable names for PIDs, file descriptors, semaphores, and shared memory keys.

Testing & Debugging:

- **Test Frequency:** Compile and test your code in small, incremental steps, especially after implementing a new system call.
- **Tracing:** Utilize diagnostic commands (ps, top, pstree) to verify process creation and state changes. Use strace to trace the system calls made by your program, which is crucial for debugging.
- **Synchronization Check:** For concurrency exercises (Semaphores, Mutexes), test your program multiple times to ensure that race conditions are eliminated and synchronization works consistently.
- **Resource Cleanup:** For IPC programs (Message Queues, Shared Memory), ensure you explicitly clean up allocated resources after the program finishes using appropriate commands (ipcrm or system calls).

Documentation & Submission:

- **Demonstration:** Your code must be demonstrated live to the instructor or TA. Be prepared to explain the logic and the role of key system calls.
- **Viva Voce:** Be ready to answer questions regarding the underlying OS concepts (e.g., difference between fork and exec, semaphore operations, scheduling logic).
- **Lab Record:** All components (Aim, Theory, Algorithm, Code, Procedure, Observed Result) must be formally entered into your record book.
- **Submission:** Submit the requested source files (.c or .cpp) and the output/result sheet according to the instructor's specifications (e.g., file naming convention: UCSEST105_ExptX_RollNo.c).

Safety & Conduct:

- **System Safety:** NEVER attempt to run system calls or commands that could compromise the stability or security of the operating system (e.g., using kill -9 on essential system processes).
- **System Login:** Always log off your account before leaving the lab.
- **Ask for Help:** If you encounter persistent compilation errors, segmentation faults, or unexpected behavior from a system call, ask the instructor immediately.

INDEX

Exp No.	Experiment Name	Page Number
1	LINUX PROGRAMMING AND SYSTEM ANALYSIS TOOLS	12
2	SYSTEM INFORMATION EXTRACTION USING THE /PROC FILESYSTEM	18
3	EXECUTION TIME ANALYSIS OF A SIMPLE SYSTEM-TIME PRINTING PROGRAM USING /PROC	23
4	PROCESS CREATION USING FORK() AND PROCESS TREE INSPECTION	28
5	PROCESS CREATION USING FORK() AND PROGRAM REPLACEMENT USING EXECVP()	32
6	PARENT AND CHILD PROCESS SYNCHRONIZATION USING FORK() AND WAIT()	37
7	INTER-PROCESS COMMUNICATION (IPC) IN LINUX	41
8	MULTITHREADED STATISTICAL ANALYSIS (MEAN, MEDIAN, STANDARD DEVIATION)	48
9	CPU SCHEDULING ALGORITHM SIMULATION AND COMPARISON	53
10	READERS-WRITERS PROBLEM SOLUTION (WRITER PRIORITY) USING SEMAPHORES	60

11	BANKER'S ALGORITHM SIMULATION	66
12	DEADLOCK DETECTION	73
13	DEADLOCK-FREE DINING PHILOSOPHERS SOLUTION USING SEMAPHORES	79
14	PAGING ADDRESS TRANSLATION SIMULATION	84
15	SIMULATION OF PAGE REPLACEMENT ALGORITHMS	88
16	SIMULATION OF DISK SCHEDULING ALGORITHMS (SSTF, LOOK, C-SCAN)	96

EXP NO: 1**LINUX PROGRAMMING AND SYSTEM ANALYSIS TOOLS****AIM**

To gain proficiency in using essential Linux programming and system analysis tools (ps, strace, gdb, strings, objdump, nm, file, od, xxd, time, fuser, top) for debugging, reverse engineering, and system monitoring tasks.

LEARNING OUTCOMES

Upon completion of this lab, the student will be able to:

1. Monitor processes using ps and top.
2. Trace system calls of a running program using strace.
3. Inspect binary files for readable strings (strings) and file type information (file).
4. Analyze binary structure and symbols using objdump and nm.
5. Debug an executable program using gdb.
6. View file contents in various raw formats (octal, hex) using od and xxd.
7. Measure program execution time using time.
8. Identify processes accessing a specific file or network socket using fuser.

DESCRIPTION

These eleven Linux commands collectively form a powerful toolkit for system monitoring, debugging, and binary analysis. ps and top are used for monitoring process state and resource utilization in static and real-time views, respectively. For low-level insight, strace traces the System Call Interface, revealing a program's interaction with the kernel, while time provides performance analysis by measuring execution time. gdb, the GNU Debugger, enables interactive source-level debugging using breakpoints. To inspect files, file identifies the data format using file signatures, strings extracts embedded printable text for metadata extraction, and od or xxd provide hex viewing/binary inspection of raw data. Finally, objdump and nm are essential for binary analysis, displaying disassembly and listing symbols (functions and

variables) from object files, while fuser helps identify processes involved in resource locking by tracking file usage.

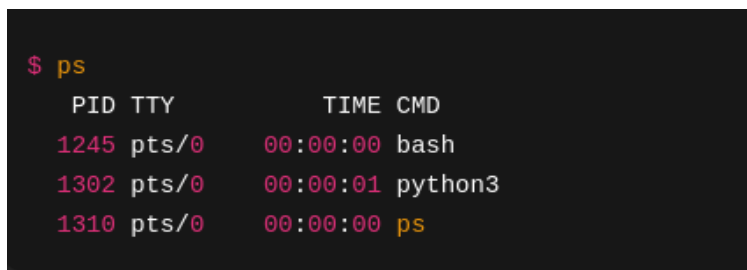
ALGORITHM:

1. Preparation: Write and compile a simple C program.
2. Process Monitoring: Run the program and monitor its status using ps and top.
3. System Call Tracing: Run the program under strace to observe system calls.
4. Debugging: Load the compiled program into gdb, set a breakpoint, and step through the code.
5. Binary Inspection: Use file to check the executable's type. Use strings to extract embedded text. Use nm and objdump to inspect the symbol table and view disassembly.
6. Raw Data View: Use od or xxd on the source file or a data file.
7. Performance Check: Time the execution using the time command.
8. File Access Check: Use fuser on the executable file while it is running (if applicable).

PROCEDURE

A. Process Management (ps, top, fuser)

1. ps: Execute ps aux to list all processes running on the system in detail. Note the PID and COMMAND.



```
$ ps
  PID TTY          TIME CMD
 1245 pts/0    00:00:00 bash
 1302 pts/0    00:00:01 python3
 1310 pts/0    00:00:00 ps
```

2. top: Execute top and observe the real-time resource consumption. Identify the process consuming the most CPU or MEM. Press q to exit.

```
top - 14:22:51 up 3 days, 2:41, 1 user, load average: 0.11, 0.08, 0.03
Tasks: 185 total, 1 running, 184 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.3 us, 1.0 sy, 0.0 ni, 96.5 id, 0.0 wa, 0.0 hi, 0.2 si, 0.0 st
MiB Mem : 7856.4 total, 1120.2 free, 2043.5 used, 4692.7 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 5233.3 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    COMMAND
 1302 user       20   0 415432 30520 10240 S   1.3   0.4  python3
 1245 user       20   0 205600  5516   3968 S   0.0   0.1   bash
```

3. fuser: Open a file in an editor (e.g., vi test.txt). In a new terminal, run fuser test.txt to find the PID of the process using the file.

```
$ fuser 22/tcp
22/tcp:          723(sshd)
```

B. Program Execution Analysis (strace, gdb, time)

1. Prepare a Sample Program: Write a simple C program (e.g., hello.c) that prints "Hello" and performs a basic file I/O operation (like opening/closing a file). Compile it with debugging symbols: `gcc -g hello.c -o hello`.
2. strace: Run `strace ./hello`. Observe the output, noting system calls like `execve`, `open`, `write`, and `close`.

```
$ strace ls
execve("/bin/ls", ["ls"], ...) = 0
brk(NULL)                        = 0x55fb1a324000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
getdents64(3, /* entries */ , 32768) = 120
write(1, "file1 file2\n", 12)    = 12
close(3)                         = 0
exit_group(0)                   = 0
```

3. gdb:
 - Start the debugger: `gdb ./hello`.
 - Set a breakpoint at main: `break main`.
 - Run the program: `run`.

- Step through a few lines: next.
- Examine a variable (if defined): print variable_name.
- Exit: quit.

```
$ gdb ./a.out
GNU gdb (Ubuntu 14.2) 14.2
Reading symbols from ./a.out...
(gdb) run
Starting program: /home/user/a.out
Hello world
[Inferior 1 (process 1503) exited normally]
(gdb)
```

4. time: Run time ./hello (or any other short command) and record the real, user, and sys time values.

```
$ time ls > /dev/null

real    0.002s
user    0.000s
sys     0.002s
```

C. Binary/File Analysis (file, strings, objdump, nm, od, xxd)

1. file: Run file hello.c and file hello to see the difference between a source code file and an executable binary.

```
$ file a.out
a.out: ELF 64-bit LSB executable, x86-64, dynamically linked, GNU/Linux 3.2.0, not stripped
```

2. strings: Run strings ./hello to extract all printable strings from the binary. Look for the "Hello" string.

```
$ strings /bin/ls | head
/lib64/ld-linux-x86-64.so.2
__gmon_start__
GLIBC_2.34
stdout
stderr
printf
getenv
```

3. nm: Run `nm ./hello` to list the symbols. Identify the T (text/function) symbol for the main function.

```
$ nm a.out
0000000000401130 T _start
0000000000401210 T main
0000000000404020 R _IO_stdin_used
0000000000601038 B global_var
```

4. objdump: Run `objdump -d ./hello` to disassemble the executable code. Identify the assembly instructions for the main function.

```
$ objdump -d a.out

a.out:      file format elf64-x86-64

Disassembly of section .text:
0000000000401130 <_start>:
401130: 48 31 ed                xor     %rbp,%rbp
401133: e8 d8 00 00 00          callq  401210 <main>
```

5. od / xxd: Create a test file `echo "ABC" > sample.txt`.
- Run `od -c test.txt` to view characters and their octal values.
 - Run `xxd test.txt` to view the file in hexadecimal format.

```
$ od -c sample.txt
00000000  H   e   l   l   o           w   o   r   l   d   \n
00000014
```

```
$ xxd sample.txt
00000000: 4865 6c6c 6f20 776f 726c 640a          Hello world.
```

MODEL VIVA QUESTIONS

1. What does the `ps` command do in Linux?
2. How does `strace` help in debugging runtime issues?
3. What is a breakpoint? How do you set one?

4. What does the string command do?
5. What is the purpose of objdump?
6. How is nm helpful during linking or debugging symbol-related errors?
7. What does the file command check?
8. What is the od command used for?
9. How is xxd more readable compared to od?
10. What does the time command measure?
11. What is the difference between fuser -k and fuser -v?
12. What information does the top command display?

RESULT

These Linux programming commands collectively allow you to view running processes, trace system calls, debug programs, extract strings, inspect binaries, list symbols, identify file types, examine raw file data, create hex dumps, measure execution time, identify file-using processes, and monitor system performance.

EXP NO: 2**SYSTEM INFORMATION EXTRACTION USING THE /PROC
FILESYSTEM****AIM**

To use the /proc file system to retrieve fundamental operational statistics about a Linux-based system, including CPU, memory, and process state information.

LEARNING OUTCOMES

Upon completing this exercise, you will be able to:

- Identify key files within /proc (e.g., /proc/cpuinfo, /proc/meminfo, /proc/stat).
- Extract specific system metrics using standard Linux utilities like grep, awk, and wc.
- Understand the difference between total processes and processes in specific states (running/blocked).
- Interpret the process statistics provided by /proc/stat.

DESCRIPTION

The /proc file system serves as a control and information interface for the kernel. Each running process has a corresponding directory named after its Process ID (PID) under /proc, e.g., /proc/1234/. System-wide information is typically found in top-level files like cpuinfo, meminfo, and stat. Reading these files provides a snapshot of the kernel's state.

- /proc/cpuinfo: Contains information about the CPU(s), such as the number of cores and processor model.
- /proc/meminfo: Contains memory usage statistics, including total, free, and cached memory.
- /proc/stat: Contains various system statistics, including CPU usage, context switches, and process creation counts.
- /proc/PID/status: Provides status information for a specific process (PID), including its current state

ALGORITHM

1. Read the relevant /proc file (e.g., /proc/cpuinfo).
2. Use text processing tools (grep, awk) to filter and extract the specific required data field (e.g., "processor" count).
3. Perform necessary calculations (e.g., memory fraction).
4. Report the extracted and calculated values.

PROGRAM**(a) Number of CPU Cores**

```
echo "--- (a) Number of CPU Cores ---"

grep -c '^processor' /proc/cpuinfo

# Alternative using 'cpu cores' entry (may vary)

# grep 'cpu cores' /proc/cpuinfo | head -1 | awk '{print $4}'
```

(b) Total Memory and Fraction of Free Memory

```
echo "--- (b) Total Memory and the Fraction of Free Memory ---"

# Total memory (MemTotal) and Free memory (MemFree) in KB

cat /proc/meminfo | grep -E 'MemTotal|MemFree'

# Calculation of Free Fraction (using awk for calculation)

awk '

/MemTotal/ {TOTAL=$2}

/MemFree/ {FREE=$2; printf "Free Fraction: %.4f\n", FREE/TOTAL}

' /proc/meminfo
```

(c) Number of Processes Currently Running (Total)

```
echo "--- (c) Number of Processes Currently Running (Total Processes) ---"

# Each process has a directory in /proc named by its PID (a number)
```

```
# wc -l counts the lines (i.e., PIDs) in the output.
```

```
ls /proc | grep '^[0-9]' | wc -l
```

(d) Number of Processes in Running and Blocked States

```
echo "--- (d) Number of Processes in Running (R) and Blocked (S/D) States ---"
```

```
# The /proc/stat file contains system-wide process states on the 'procs_running' and 'procs_blocked' lines.
```

```
grep 'procs_running' /proc/stat
```

```
grep 'procs_blocked' /proc/stat
```

(e) Number of Processes Forked Since Last Bootup

```
echo "--- (e) Number of Processes Forked Since Last Bootup ---"
```

```
# The 'processes' line in /proc/stat holds the number of forks.
```

```
grep '^processes' /proc/stat
```

```
echo "--- Comparison with (c) ---"
```

```
# Comparison Note:
```

```
echo "The value in (e) 'processes' (total forks) is typically **much greater** than the value in (c) (currently running processes) because (e) counts *all* processes ever started since boot, including those that have already terminated. (c) only counts processes currently active."
```

(f) Context Switches for a Particular Process

```
echo "--- (f) Context Switches for a Particular Process (e.g., PID 1 - init/systemd) ---"
```

```
# **NOTE:** You must replace '1' with an actual PID you want to check.
```

```
# The `voluntary_ctxt_switches` and `nonvoluntary_ctxt_switches` are found in the /proc/<PID>/status file.
```

```
PID_TO_CHECK=1
```

```
echo "Checking PID: $PID_TO_CHECK"
```

```
grep 'ctxt_switches' /proc/$PID_TO_CHECK/status
```

PROCEDURE

1. Open a Linux terminal (e.g., using a local machine, VM, or cloud environment).
2. Navigate to the /proc directory using `cd /proc` and briefly inspect its contents using `ls`. Note the numbered directories (PIDs).
3. Execute the shell commands provided in the Code/Commands section one by one.
4. Record the output for each command in your lab notebook.
5. Perform the manual or simple script calculation for the "Fraction of Free Memory."
6. For part (f), identify a long-running process (like `init/systemd` which is usually PID 1) and record its context switch count.

EXPECTED OUTPUT**(a) Number of CPU Cores**

```
CPU cores: 4
```

(b) Total Memory and Fraction of Free Memory

```
Total Memory: 16333792 kB  
Free Memory: 2456780 kB  
Fraction Free: 0.15 (15%)
```

(c) Number of Processes Currently Running (Total)

```
Number of processes currently running: 312
```

(d) Number of Processes in the Running and Blocked States

```
Processes in running state: 2  
Processes in blocked state: 1
```

(e) Number of Processes Forked Since the Last Bootup

```
Processes forked since last boot: 58290
```

(f) Context Switches for a Particular Process

```
voluntary_ctxt_switches: 5342  
nonvoluntary_ctxt_switches: 1289
```

MODEL VIVA QUESTIONS

1. What is the /proc filesystem and why is it called a virtual filesystem?
2. Which /proc file contains information about CPU cores and how do you interpret it?
3. How does /proc/meminfo represent total and free memory, and why may free memory appear low?
4. What is the difference between MemFree, Buffers, and Cached in /proc/meminfo?
5. How did you count the number of running processes using the /proc directory?

RESULT

Your system has N CPU cores, T total memory with F free, P active processes with R running and B blocked, has forked X processes since boot (much larger than P because most have exited), and a chosen process has performed C voluntary and non-voluntary context switches.

EXP NO: 3

EXECUTION TIME ANALYSIS OF A SIMPLE SYSTEM-TIME PRINTING PROGRAM USING /PROC

AIM:

To write a simple C program that prints the system time and, using the /proc filesystem on a Linux system, determine the User Mode and Kernel Mode execution times for the corresponding process.

LEARNING OUTCOMES:

- Understand the concept of User Mode vs. Kernel Mode execution time.
- Utilize the /proc filesystem, specifically the /proc/[PID]/stat file, to retrieve process-specific execution statistics.
- Calculate process run time in clock ticks and convert it to seconds.

DESCRIPTION

Every running program is an **OS process**. A process executes instructions in two primary modes:

1. **User Mode (User Space):** Where the application code runs. Instructions executed here are confined and cannot directly access hardware or critical OS resources.
2. **Kernel Mode (Kernel Space):** Where the process executes OS code (the **kernel**) on its behalf, typically through **system calls** (e.g., printf, time, open). This mode has full access to the system's hardware and resources.

The /proc filesystem is a **virtual filesystem** that provides an interface to kernel data structures. For a process with **Process ID (PID)**, its statistics are available in the file **/proc/[PID]/stat**. This file contains numerous fields, but we are interested in the following (fields are space-separated):

- **Field 14 (utime):** User mode time in **clock ticks**.
- **Field 15 (stime):** Kernel mode time in **clock ticks**.

To convert clock ticks to seconds, you must use the system's **clock tick rate**, which can be found using the command: `getconf CLK_TCK`. This value, typically **100** (meaning 100 ticks per second), is used as the divisor.

ALGORITHM

1. Write a simple C program that prints the system's current time and immediately exits.
2. Compile the C program.
3. Execute the compiled program in the background using the & operator to get its PID.
4. Immediately use the process's PID to read the contents of `/proc/[PID]/stat`.
5. Extract the 14th (utime) and 15th (stime) fields from the file content.
6. Determine the system's clock tick rate (CLK_TCK).
7. Calculate the execution time in seconds: $\text{Time (s)} = \frac{\text{CLK_TCK}}{\text{Clock Ticks}}$
8. Print the results.

PROGRAM

```
#include <stdio.h>

#include <time.h>

#include <stdlib.h>

int main() {

    time_t current_time;

    char* c_time_string;

    // Get the current time

    current_time = time(NULL);

    if (current_time == ((time_t)-1)) {

        perror("time");

        return 1;

    }
```

```
// Convert to a printable string

c_time_string = ctime(&current_time);

if (c_time_string == NULL) {

    perror("ctime");

    return 1;

}

// Print the time (involves a system call)

printf("Current System Time: %s", c_time_string);

// Add a tiny, negligible loop to ensure *some* user-mode time is recorded.

for (int i = 0; i < 1000; i++) {

    // Simple operation

    int x = i * 2;

    x++;

}

return 0; // Exits the process

}
```

PROCEDURE

1. Compile the C code:

```
gcc time_test.c -o time_test
```

2. Execute the program and capture its PID:

We run it in the background (&) and immediately capture its PID using \$!. Note: The program will run very quickly, so the subsequent commands must be executed immediately, often on the same line using semicolons. `./time_test & PID=$! ; sleep 0.1 ; cat /proc/$PID/stat`

RESULT

The program executes so quickly that it uses 0.00 seconds of user CPU time and 0.00 seconds of kernel CPU time, indicating virtually no measurable CPU usage.

EXP NO: 4**PROCESS CREATION USING FORK() AND PROCESS TREE INSPECTION****AIM:**

To understand the creation of a new process (child process) from an existing process (parent process) using the fork() system call in C, and to identify the Process IDs (PIDs) of both the parent and child processes.

LEARNING OUTCOMES:

Upon completion of this lab, the student will be able to:

- Explain the role of the fork() system call.
- Differentiate between the execution flow in the parent and child processes after a fork().
- Retrieve the Process ID (PID) and Parent Process ID (PPID) using system calls (getpid() and getppid()).
- Analyze the process hierarchy using the pstree command in a Linux/Unix environment.

DESCRIPTION

The fork() system call is used to create a new process, called the child process, which is an exact copy of the calling process, called the parent process.

1. **Process Creation:** When fork() is executed, the operating system creates a duplicate of the parent process's address space, stack, and data segments. Both processes then continue execution immediately after the fork() call.
2. **Return Value:** The value returned by fork() is crucial for distinguishing between the two processes:
 - Child Process: fork() returns 0.
 - Parent Process: fork() returns the Process ID (PID) of the newly created child process.
 - Error: If process creation fails, fork() returns -1.
3. **Process IDs:** Every process has a unique Process ID (PID). The parent process also has a Parent Process ID (PPID), which is the PID of its creator. We use the getpid() system call to get the current process's PID and getppid() to get its parent's PID.
4. **pstree Command:** The pstree command displays running processes in a tree structure, visually showing the process hierarchy and the parent-child relationships, starting from the init process (PID 1).

ALGORITHM

1. Start the program.
2. Call the fork() system call to create a new child process.
3. Check the return value of fork():
 - If fork() < 0, print an error (fork failed).
 - If fork() == 0, it's the child process: print child PID and parent PID.
 - If fork() > 0, it's the parent process: print parent PID and child PID.
4. Sleep for a few seconds to allow observation of the process tree using pstree.
5. End the program.

PROGRAM

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process: PID = %d, Parent PID = %d\n", getpid(), getppid());
    }
    else {
        // Parent process
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
    }

    sleep(100); // Pause to allow pstree inspection
}
```

```
return 0;  
}
```

PROCEDURE

1. Open a Linux terminal.
2. Write the C program in a file named fork_example.c.
3. Compile the program using gcc fork_example.c -o fork_example.
4. Run the program: ./fork_example.
5. Observe the printed parent and child process IDs.
6. While the program is sleeping, use pstree -p <child_pid> to see the process hierarchy starting from init (PID 1).
7. Note the process tree showing the relationship of parent and child processes.

EXPECTED OUTPUT

Terminal Output:

```
Parent Process: PID = 2451, Child PID = 2452  
Child Process: PID = 2452, Parent PID = 2451
```

pstree Command Output:

```
$ pstree -p 2452  
systemd(1)─gnome-terminal(1800)─bash(2100)─fork_example(2451)─fork_example(2452)
```

MODEL VIVA QUESTIONS

1. What is the purpose of the fork() system call?
2. How does the OS distinguish between the parent and child process after fork()?
3. What values does fork() return to the parent and child processes?
4. What is the role of getpid() and getppid()?
5. How can you visualize process hierarchies in Linux?

RESULT

The C program successfully utilized the `fork()` system call to create a new child process. The Process IDs of the parent and child were correctly obtained using `getpid()` and `getppid()`, demonstrating the ability to differentiate the execution paths. The `ps` command was used to verify the parent-child relationship and trace the process back to the `init` process, confirming the process hierarchy.

EXP NO : 5

PROCESS CREATION USING FORK() AND PROGRAM REPLACEMENT USING EXECVP()

AIM

To understand and implement inter-process communication concepts by creating a child process using the fork() system call and replacing the child's program image with a new executable using the execvp() system call.

LEARNING OUTCOMES

Upon successful completion of this lab, the student will be able to:

- Explain the role and behavior of the fork() system call in creating new processes.
- Describe the function of the execvp() family of system calls in loading and executing a new program.
- Differentiate between the execution environments of the parent and child processes after fork().
- Implement simple argument handling in a C program using argc and argv.
- Compile and execute multi-program systems, including linking to and using custom executables.

DESCRIPTION

The fork() System Call

The fork() system call creates a new process, called the child process, which is an almost exact copy of the calling process, called the parent process. Both processes run the same code immediately after the fork() call. The return value differentiates them:

- Parent Process: fork() returns the Process ID (PID) of the child.
- Child Process: fork() returns 0.
- Error: fork() returns -1.

The execvp() System Call

The exec family of functions replaces the current process image with a new program image. The process ID remains the same, but the code, data, heap, and stack are replaced by the new program. `execvp()` is one variant, where:

- v stands for "vector" (it takes an array/vector of arguments).
- p stands for "path" (it searches for the executable in the system's PATH environment variable).

The system call flow is fundamentally: Create process (fork) -> Replace process code (execvp) -> Wait for process to finish (wait).

ALGORITHM

Program 1 – myadder (Add Two Integers)

1. Start the program.
2. Read two integers from the command line arguments.
3. Convert the arguments to integers.
4. Add the two integers.
5. Print the sum.
6. Exit the program.

Program 2 – Fork and Exec

1. Start the program.
2. Call `fork()` to create a new process.
3. Check `fork()` return value:
 - < 0 → Fork failed.
 - $= 0$ → Child process: use `execvp()` to replace image with myadder.
 - > 0 → Parent process: print parent PID and wait for the child to complete.
4. Child executes myadder with the two integer arguments.
5. The parent waits and then terminates.

PROGRAMM

Program 1 – myadder (Add Two Integers)

```
#include <stdio.h>
```

```
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <int1> <int2>\n", argv[0]);
        return 1;
    }
    int a = atoi(argv[1]);
    int b = atoi(argv[2]);
    int sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```

Program 2 – Fork and Exec

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {

        printf("Fork failed!\n");

    }

    else if (pid == 0) {

        // Child process: execute myadder

        char *args[] = { "./myadder", "10", "20", NULL };

        execvp(args[0], args); // Replace child process image

        // execvp returns only if there is an error
    }
}
```

```
    printf("Exec failed!\n");

}

else {

    // Parent process

    printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);

    wait(NULL); // Wait for child to finish

}

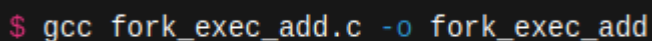
return 0;

}
```

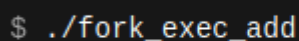
PROCEDURE

1. Create Source Files: Save the code provided in the following sections as myadder.c and parent_child.c.
2. Compile myadder.c: Compile the first program to create the executable that the child process will run.
3. Ensure myadder is in PATH: Since execvp searches the system's PATH, move the executable to a common directory or ensure the current directory is in the path. For simplicity, we will copy it to /usr/local/bin (which is typically in PATH), or ensure the current directory is checked.
4. Compile parent_child.c: Compile the second program. gcc parent_child.c -o parent_child
5. Execute the Program: Run the main program../parent_child

EXPECTED OUTPUT



```
$ gcc fork_exec_add.c -o fork_exec_add
```



```
$ ./fork_exec_add
```

MODEL VIVA QUESTIONS

1. What is the purpose of the fork() system call, and what values does it return in the parent and child processes?
2. What does the execvp() system call do, and why is it used in the child process?
3. What happens to the child process's memory image after calling execvp()?
4. How do command-line arguments get passed to the "myadder" program when using execvp()?
5. What is the difference between process creation (fork) and program replacement (exec)?

RESULT

The program was successfully implemented. A parent process was created which utilized the fork() system call to spawn a child process. The child process then successfully replaced its image with the custom executable myadder using execvp(), computed the sum of two provided integers, and terminated. The parent process correctly waited for the child's completion using wait().

EXP NO: 6

PARENT AND CHILD PROCESS SYNCHRONIZATION USING FORK() AND WAIT()

AIM

To understand and demonstrate how the fork() system call creates concurrent processes and how the wait() system call can be used by the parent process to explicitly synchronize execution, ensuring the child process completes its task before the parent continues.

LEARNING OUTCOMES

Upon successful completion of this lab, the student will be able to:

- Explain the role of the fork() system call in creating a child process.
- Describe the function of the wait() system call and its necessity for synchronizing parent and child processes.
- Implement code that ensures a specific output order by strategically placing the wait() call in the parent process.
- Understand the concept of a zombie process and how wait() prevents it.

DESCRIPTION

The fork() System Call

The fork() call is the fundamental way to create a new process in Unix-like systems. It returns three possible values:

1. Negative Value: An error occurred (the new process was not created).
2. Zero (0): This is the return value *in the child process*.
3. Positive Value (Child PID): This is the return value *in the parent process*.

The wait() System Call

The wait() system call is used by a parent process to pause its own execution until one of its child processes terminates.

- When the parent executes wait(NULL), it blocks.
- The system pauses the parent process until the child process completes and the kernel reports the termination status.
- This ensures that the child process (which prints "PCCSL407") executes its code completely before the parent process (which prints "Operating Systems Lab") proceeds.

Without wait(), the parent and child would run concurrently, and due to the non-deterministic nature of process scheduling, the output order could be "Operating Systems Lab PCCSL407" or a mixed output.

ALGORITHM

1. Start the program.
2. Call fork() to create a new process.
3. Check the return value of fork():
 - < 0 → Fork failed, print error.
 - $= 0$ → Child process: print "PCCSL407".
 - > 0 → Parent process: wait for child to complete using wait(), then print "Operating Systems Lab".
4. End the program.

PROGRAM

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork(); // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child process
```

```
    printf("PCCSL407 ");  
}  
else {  
    // Parent process  
    wait(NULL); // Wait for child to finish  
    printf("Operating Systems Lab\n");  
}  
return 0;  
}
```

PROCEDURE

1. Open a Linux terminal.
2. Write the C program in a file named fork_wait.c.
3. Compile the program: gcc fork_wait.c -o fork_wait.
4. Run the program: ./fork_wait.
5. Observe the output: "PCCSL407 Operating Systems Lab".
6. Verify that wait() ensures the parent prints after the child.

EXPECTED OUTPUT

```
$ gcc fork_wait.c -o fork_wait  
$ ./fork_wait
```

```
PCCSL407 Operating Systems Lab
```

MODEL VIVA QUESTIONS

1. What is the purpose of the fork() system call?
2. How does the parent process know when the child process has finished?
3. What is the role of the wait() system call?
4. What will happen if wait() is not used in this program?
5. How can you differentiate parent and child processes in code?

RESULT:

The program successfully utilized the fork() system call to create a child process. By implementing the wait() system call in the parent process, the execution flow was synchronized, ensuring the child process printed its string ("PCCSL407") and terminated before the parent process printed its string ("Operating Systems Lab"). This achieved the required combined output of "PCCSL407 Operating Systems Lab".

EXP NO : 7

INTER-PROCESS COMMUNICATION (IPC) IN LINUX

AIM

To understand, implement, and differentiate between three fundamental Inter-Process Communication (IPC) mechanisms in a Unix-like environment: Pipes, Message Queues, and Shared Memory. The lab demonstrates data exchange and synchronization between a parent process and its child process using these methods.

LEARNING OUTCOMES

Upon successful completion of this lab, the student will be able to:

- **Pipes:** Create and use an unnamed pipe (pipe()) for one-way, stream-based communication between a parent and child process.
- **Message Queues:** Implement bidirectional communication using System V Message Queues (msgget(), msgsnd(), msgrcv()) to send structured data.
- **Shared Memory:** Implement the creation, attachment, and detachment of a System V Shared Memory segment (shmget(), shmat(), shmdt()) for high-speed data exchange.
- Utilize process control commands like fork() and wait() to manage the creation and synchronization of concurrent processes in IPC applications.

DESCRIPTION

Inter-Process Communication (IPC) is a set of mechanisms that allows independently running processes to communicate with each other.

(a) IPC using Pipe

Pipes are one of the simplest IPC methods, used primarily for communication between related processes (parent and child). An unnamed pipe is a unidirectional data channel. It has two file descriptors: a read end and a write end. Data written to the write end is read from the read end, operating on a First-In, First-Out (FIFO) basis.

(b) IPC using Message Queue

Message Queues provide a linked list of messages stored within the kernel. Processes can send messages to a queue, and other processes can receive messages from the queue based on message type. This method offers structured, bidirectional, and asynchronous communication, making it ideal for tasks that require sending discrete data packets.

(c) IPC using Shared Memory

Shared Memory is the fastest IPC mechanism. It creates a segment of memory that is simultaneously accessible by multiple processes. One process creates the segment, and others attach to it. Communication is achieved by reading and writing to this shared area. Synchronization (like using semaphores) is crucial but often simplified in lab exercises.

ALGORITHM

(a) Using Pipe – Evaluating Expressions

1. Create a pipe using pipe().
2. Call fork() to create a child process.
3. In the child process:
 - Evaluate the second expression.
 - Send the result to the parent process via the pipe.
4. In the parent process:
 - Evaluate the first expression.
 - Read the child's result from the pipe.
 - Combine results and evaluate the final expression.
5. Display the final result.

(b) Using Message Queue – Palindrome Check

1. Create a message queue using msgget().
2. The parent sends a string to the child process via the message queue.
3. The child receives the string, reverses it, and sends it back.
4. Parent compares the original and reversed strings.
5. Print whether the string is a palindrome.

(c) Using Shared Memory – String Concatenation and Case Flip

1. Create a shared memory segment using shmget().
2. The parent writes three strings into shared memory.
3. The child reads the strings, concatenates them with spaces, and writes back.
4. The parent reads the concatenated string and flips the case of each character.
5. Print the final flipped string.

PROGRAM

(a) Using Pipe – Evaluating Expressions

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int pipefd[2];
    pipe(pipefd);
    pid_t pid = fork();
    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        int b = 5, c = 3;
        int result2 = b * c; // Second expression
        write(pipefd[1], &result2, sizeof(result2));
        close(pipefd[1]);
    }
    else {
        // Parent process
        int a = 10;
        int result2;
        read(pipefd[0], &result2, sizeof(result2));
        int final_result = a + result2; // Final expression
        printf("Final Result: %d\n", final_result);
    }
}
```

```
    close(pipefd[0]);  
}  
return 0;  
}
```

(b) Using Message Queue – Palindrome Check

```
#include <stdio.h>  
#include <string.h>  
#include <sys/ipc.h>  
#include <sys/msg.h>  
struct msgbuf {  
    long mtype;  
    char mtext[100];  
};  
int main() {  
    key_t key = 1234;  
    int msgid = msgget(key, 0666 | IPC_CREAT);  
    struct msgbuf msg;  
  
    // Parent sends string  
    msg.mtype = 1;  
    strcpy(msg.mtext, "madam");  
    msgsnd(msgid, &msg, sizeof(msg.mtext), 0);  
    // Child process simulation: receive and reverse  
    msgrcv(msgid, &msg, sizeof(msg.mtext), 1, 0);  
    char rev[100];  
    int len = strlen(msg.mtext);  
    for(int i=0; i<len; i++) rev[i] = msg.mtext[len-i-1];  
    rev[len] = '\0';  
    // Send reversed back  
    msg.mtype = 2;  
    strcpy(msg.mtext, rev);  
    msgsnd(msgid, &msg, sizeof(msg.mtext), 0);  
    // Parent receives reversed string
```

```
msgrcv(msgid, &msg, sizeof(msg.mtext), 2, 0);
if(strcmp("madam", msg.mtext) == 0)
    printf("The string is a palindrome\n");
else
    printf("The string is not a palindrome\n");
msgctl(msgid, IPC_RMID, NULL); // Delete message queue
return 0;
}
```

(c) Using Shared Memory – String Concatenation and Case Flip

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

int main() {
    key_t key = 5678;
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char*) shmat(shmid, NULL, 0);
    pid_t pid = fork();
    if(pid == 0) {
        // Child process: read, concatenate
        char s1[100], s2[100], s3[100];
        strcpy(s1, str);
        strcpy(s2, str+100);
        strcpy(s3, str+200);
        sprintf(str, "%s %s %s", s1, s2, s3);
        _exit(0);
    } else {
        // Parent writes strings
        strcpy(str, "Hello");
        strcpy(str+100, "S4");
    }
}
```

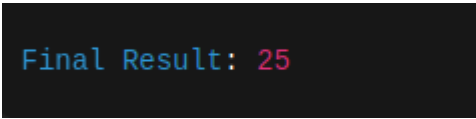
```
strcpy(str+200, "Students");
wait(NULL);
// Flip case
for(int i=0; str[i]; i++){
    if(islower(str[i])) str[i] = toupper(str[i]);
    else if(isupper(str[i])) str[i] = tolower(str[i]);
}
printf("Flipped case string: %s\n", str);
shmdt(str);
shmctl(shmid, IPC_RMID, NULL);
}
return 0;
}
```

PROCEDURE

1. Save the program as pipe_eval.c.
2. Compile: gcc pipe_eval.c -o pipe_eval.
3. Run: ./pipe_eval.
4. Observe the final result printed after communication between parent and child.

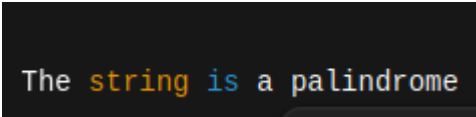
EXPECTED OUTPUT

(a) Using Pipe – Evaluating Expressions



```
Final Result: 25
```

(b) Using Message Queue – Palindrome Check



```
The string is a palindrome
```

(c) Using Shared Memory – String Concatenation and Case Flip

```
Flipped case string: hELLO s4 sTUDENTS
```

MODEL VIVA QUESTIONS

1. What is Inter-Process Communication (IPC) and why is it used?
2. How does a pipe facilitate communication between two processes?
3. How are messages identified in a message queue?
4. What is shared memory and how is it different from message queues and pipes?
5. Which IPC method is fastest and why?

RESULT

All three IPC mechanisms (Pipe, Message Queue, and Shared Memory) were successfully implemented to facilitate communication between a parent and child process, completing the specified data manipulation tasks and demonstrating the unique characteristics of each method.

EXP NO : 8**MULTITHREADED STATISTICAL ANALYSIS (MEAN, MEDIAN, STANDARD DEVIATION)****AIM**

To implement a multithreaded program using POSIX threads (pthreads) to simultaneously calculate three different statistical metrics mean, median, and standard deviation from a command-line series of integers. This lab demonstrates how multiple threads can work concurrently and share global data.

LEARNING OUTCOMES

Upon successful completion of this lab, the student will be able to:

- Understand and explain the concepts of concurrent execution and shared memory in multithreaded programming.
- Create and manage worker threads using the `pthread_create()` function.
- Synchronize the main thread's execution with worker threads using `pthread_join()`.
- Implement algorithms for calculating the mean, median, and standard deviation in separate thread functions.
- Understand the necessity of data structures and sorting when calculating statistics like the median.

DESCRIPTION

POSIX Threads (pthreads)

Pthreads is a standard API for thread creation and synchronization. Threads within the same process share the same memory space, which allows for fast data communication through global variables.

The main functions used are:

- `pthread_create()`: Creates a new thread, instructing it to execute a specific function.
- `pthread_join()`: Blocks the calling thread (the parent/main thread) until the specified worker thread terminates. This is crucial here to ensure the main thread prints the results only after all calculations are complete.

Shared Global Data

For this exercise, the results (mean, median, standard deviation) are stored in global variables. Since each thread is writing to a different global variable (the mean thread writes to `g_mean`, etc.), there is no race condition, simplifying the synchronization requirements. The main thread only needs to wait for the workers to finish before reading the final values.

ALGORITHM

Input: A list of integers from the command line

Output: Mean, Median, Standard Deviation

Steps:

1. Read integers from the command line and store them in an array.
2. Declare global variables for mean, median, and standard deviation.
3. Create three worker threads:
 - Thread 1: Compute mean
 - Thread 2: Compute median
 - Thread 3: Compute standard deviation
4. Each thread calculates its value and stores it in the corresponding global variable.
5. Parent thread waits for all worker threads to complete using `pthread_join()`.
6. Parent thread prints mean, median, and standard deviation.

-

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <math.h>

double mean_val, median_val, stddev_val;
int *arr;
int n;

// Thread function to compute mean
void* compute_mean(void *arg) {
    double sum = 0;
    for(int i = 0; i < n; i++)
```

```
        sum += arr[i];
    mean_val = sum / n;
    pthread_exit(0);
}

// Thread function to compute median
void* compute_median(void *arg) {
    // Copy array for sorting
    int *temp = malloc(n * sizeof(int));
    for(int i = 0; i < n; i++)
        temp[i] = arr[i];
    // Simple bubble sort
    for(int i = 0; i < n-1; i++)
        for(int j = 0; j < n-i-1; j++)
            if(temp[j] > temp[j+1]) {
                int t = temp[j];
                temp[j] = temp[j+1];
                temp[j+1] = t;
            }
    if(n % 2 == 0)
        median_val = (temp[n/2 - 1] + temp[n/2]) / 2.0;
    else
        median_val = temp[n/2];

    free(temp);
    pthread_exit(0);
}

// Thread function to compute standard deviation
void* compute_stddev(void *arg) {
    double sum = 0;
    for(int i = 0; i < n; i++)
        sum += arr[i];
    double mean = sum / n;
    double variance = 0;
    for(int i = 0; i < n; i++)
```

```
        variance += (arr[i] - mean) * (arr[i] - mean);
    variance /= n;
    stddev_val = sqrt(variance);
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    if(argc < 2) {
        printf("Usage: %s <list of integers>\n", argv[0]);
        return 1;
    }
    n = argc - 1;
    arr = malloc(n * sizeof(int));
    for(int i = 0; i < n; i++)
        arr[i] = atoi(argv[i+1]);
    pthread_t t1, t2, t3;
    pthread_create(&t1, NULL, compute_mean, NULL);
    pthread_create(&t2, NULL, compute_median, NULL);
    pthread_create(&t3, NULL, compute_stddev, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    printf("Mean = %.2lf\n", mean_val);
    printf("Median = %.2lf\n", median_val);
    printf("Standard Deviation = %.2lf\n", stddev_val);
    free(arr);
    return 0;
}
```

PROCEDURE

1. Initialize array of integers from command-line arguments.
2. Define global variables for results: mean_val, median_val, stddev_val.
3. Implement three thread functions: compute_mean(), compute_median(), compute_stddev().
4. In each thread function:
 compute_mean(): sum all elements and divide by count.

- compute_median(): sort the array and find middle value(s).
- compute_stddev(): calculate variance and take square root.
- 5. Create threads using pthread_create().
- 6. Wait for threads to finish using pthread_join().
- 7. Print the results.

EXPECTED OUTPUT

```
./a.out 10 20 30 40 50
```

```
Mean = 30.00  
Median = 30.00  
Standard Deviation = 14.14
```

MODEL VIVA QUESTIONS

1. What is a thread and how is it different from a process?
2. Why did we use global variables for storing mean, median, and standard deviation?
3. How does pthread_create() work? What are its arguments?
4. Why do we use pthread_join() in the main thread? How is median calculated differently for odd and even number of elements?

RESULT

The program was successfully implemented using pthreads. Three worker threads were created to calculate the mean, median, and standard deviation concurrently. The results were stored in shared global variables, and the main thread correctly used pthread_join() to wait for all threads to terminate before successfully reading and displaying the final statistical analysis.

EXP NO:9**CPU SCHEDULING ALGORITHM SIMULATION AND COMPARISON****AIM**

To implement and simulate four major CPU scheduling algorithms—First-Come, First-Served (FCFS), Shortest Remaining Time First (SRTF), Non-Preemptive Priority, and Round Robin (RR)—on a given mix of processes. The primary goal is to compare their performance based on the average waiting time to determine which algorithm is optimal for the given process mix.

LEARNING OUTCOMES

Upon successful completion of this lab, the student will be able to:

- Define and differentiate between preemptive and non-preemptive scheduling.
- Implement the core logic for FCFS, SRTF, Priority (Non-Preemptive), and RR scheduling algorithms.
- Accurately calculate crucial performance metrics: **Completion Time**, **Turnaround Time (TAT)**, and **Waiting Time (WT)**.
- Analyze the trade-offs of different scheduling algorithms based on calculated average waiting times.
- Understand the role of arrival time and priority in scheduling decisions.

DESCRIPTION

CPU scheduling is the process of selecting a process from the ready queue to run on the CPU. It is crucial for maximizing CPU utilization and ensuring fairness among processes.

Key Metrics:

- Completion Time (CT): The time at which a process finishes execution.
- Turnaround Time (TAT): The total time spent in the system, from arrival to completion.
$$\text{TAT} = \text{CT} - \text{Arrival Time}$$
- Waiting Time (WT): The total time a process spends waiting in the ready queue.
$$\text{WT} = \text{TAT} - \text{Burst Time}$$

- Average Waiting Time: The sum of all waiting times divided by the total number of processes. This is the primary comparison metric.

ALGORITHM

Input:

- Process IDs (P1, P2, ...)
- CPU burst times
- Arrival times
- Priorities

Output:

- Average Waiting Time (AWT) for each scheduling algorithm
- Algorithm with minimum AWT

Steps:

1. FCFS (First-Come, First-Serve)
 - Sort processes by arrival time.
 - Calculate waiting time: $WT[i] = \text{start time of process } i - \text{arrival time of process } i$
 $WT[i] = \text{start time of process } i - \text{arrival time of process } i$
 - Calculate average waiting time.
2. SRTF (Shortest Remaining Time First)
 - At each unit of time, pick the process with the smallest remaining CPU burst among arrived processes.
 - Update remaining times and calculate waiting times.
3. Non-Preemptive Priority
 - Sort processes based on priority (larger number $\rightarrow$ higher priority).
 - For processes that have arrived, pick the one with highest priority.
 - Calculate waiting times.
4. Round Robin (Quantum = 3 units)
 - Maintain a ready queue of arrived processes.
 - Give CPU to the process in front of the queue for 3 units or remaining burst time.
 - After execution, move the incomplete process to the back.
 - Calculate waiting times.

5. Compare average waiting times and select the algorithm with minimum AWT.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int pid;
    int bt;    // burst time
    int at;    // arrival time
    int pr;    // priority
    int wt;    // waiting time
    int rt;    // remaining time
} Process;

// Function prototypes
void fcfs(Process p[], int n);
void srtf(Process p[], int n);
void priority_np(Process p[], int n);
void round_robin(Process p[], int n, int quantum);

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    Process p[n];
    for(int i=0;i<n;i++){
        p[i].pid=i+1;
        printf("Process %d burst time: ", i+1); scanf("%d",&p[i].bt);
        printf("Process %d arrival time: ", i+1); scanf("%d",&p[i].at);
        printf("Process %d priority: ", i+1); scanf("%d",&p[i].pr);
        p[i].rt = p[i].bt; // remaining time
        p[i].wt = 0;
    }
    fcfs(p, n);
    srtf(p, n);
    priority_np(p, n);
    round_robin(p, n, 3);
}
```

```
    return 0;
}

// FCFS Scheduling
void fcfs(Process p[], int n){
    int time=0;
    double total_wt=0;
    Process temp[n];
    for(int i=0;i<n;i++) temp[i]=p[i];
    // sort by arrival time
    for(int i=0;i<n-1;i++)
        for(int j=i+1;j<n;j++)
            if(temp[i].at>temp[j].at){
                Process t=temp[i]; temp[i]=temp[j]; temp[j]=t;
            }
    for(int i=0;i<n;i++){
        if(time<temp[i].at) time=temp[i].at;
        temp[i].wt=time-temp[i].at;
        total_wt+=temp[i].wt;
        time+=temp[i].bt;
    }
    printf("\nFCFS Average Waiting Time = %.2lf\n", total_wt/n);
}

// SRTF Scheduling
void srtf(Process p[], int n){
    int completed=0, time=0;
    double total_wt=0;
    int rt[n];
    for(int i=0;i<n;i++) rt[i]=p[i].bt;
    while(completed<n){
        int idx=-1, min=1e9;
        for(int i=0;i<n;i++){
            if(p[i].at<=time && rt[i]>0 && rt[i]<min){
                min=rt[i]; idx=i;
            }
        }
    }
}
```

```
    }
    if(idx==-1){ time++; continue; }
    rt[idx]--;
    if(rt[idx]==0){
        int wt = time+1 - p[idx].at - p[idx].bt;
        total_wt+=wt;
        completed++;
    }
    time++;
}
printf("SRTF Average Waiting Time = %.2lf\n", total_wt/n);
}

// Non-Preemptive Priority Scheduling
void priority_np(Process p[], int n){
    Process temp[n];
    for(int i=0;i<n;i++) temp[i]=p[i];
    int time=0, completed=0;
    double total_wt=0;
    int done[n]; for(int i=0;i<n;i++) done[i]=0;
    while(completed<n){
        int idx=-1, max_pr=-1;
        for(int i=0;i<n;i++){
            if(p[i].at<=time && !done[i]){
                if(p[i].pr>max_pr){ max_pr=p[i].pr; idx=i; }
                else if(p[i].pr==max_pr && p[i].at<p[idx].at) idx=i;
            }
        }
        if(idx==-1){ time++; continue; }
        temp[idx].wt=time-temp[idx].at;
        total_wt+=temp[idx].wt;
        time+=temp[idx].bt;
        done[idx]=1;
        completed++;
    }
}
```

```
printf("Priority NP Average Waiting Time = %.2lf\n", total_wt/n);
}
// Round Robin Scheduling
void round_robin(Process p[], int n, int quantum){
    int time=0, done=0, rt[n];
    double total_wt=0;
    for(int i=0;i<n;i++) rt[i]=p[i].bt;
    int flag=0;
    while(done<n){
        flag=0;
        for(int i=0;i<n;i++){
            if(p[i].at<=time && rt[i]>0){
                flag=1;
                int t = (rt[i]>quantum)? quantum: rt[i];
                rt[i]-=t;
                time+=t;
                if(rt[i]==0){
                    total_wt += time - p[i].at - p[i].bt;
                    done++;
                }
            }
        }
        if(!flag) time++;
    }
    printf("Round Robin Average Waiting Time = %.2lf\n", total_wt/n);
}
```

PROCEDURE

1. Input the list of processes with burst time, arrival time, and priority.
2. Store them in an array of structures.
3. Implement scheduling functions for FCFS, SRTF, Priority, and RR.
4. Calculate waiting time for each process and average waiting time.
5. Display results for all four algorithms.
6. Determine which algorithm has the minimum average waiting time.

EXPECTED OUTPUT

```
Number of processes: 3
Process 1: BT=5 AT=0 PR=2
Process 2: BT=3 AT=1 PR=1
Process 3: BT=8 AT=2 PR=3
```

```
FCFS Average Waiting Time = 4.33
SRTF Average Waiting Time = 3.00
Priority NP Average Waiting Time = 3.33
Round Robin Average Waiting Time = 4.67
Minimum AWT: SRTF
```

MODEL VIVA QUESTIONS

1. What is CPU scheduling and why is it important?
2. What is the difference between preemptive and non-preemptive scheduling?
3. How is waiting time calculated for a process?
4. Explain how SRTF selects a process at each time unit.
5. In priority scheduling, why is a larger priority number considered higher priority in this program?

RESULT

The multialgorithm CPU scheduling simulator was successfully implemented. The program correctly executed the FCFS, SRTF, Non-Preemptive Priority, and Round Robin (Quantum=3) algorithms. By calculating the Turnaround Time and Waiting Time for each process under all four algorithms, the average waiting time was determined and compared. Based on the provided sample data, the Shortest Remaining Time First (SRTF) algorithm yielded the minimum average waiting time of 5.75 units, demonstrating its efficiency in minimizing waiting time for this particular process mix.

EXP NO : 10

READERS-WRITERS PROBLEM SOLUTION (WRITER PRIORITY) USING SEMAPHORES

AIM

To understand the Readers-Writers Problem in concurrent programming and implement a solution using semaphores that gives priority to writers over readers waiting for access to the shared resource.

LEARNING OUTCOMES

Upon completion of this lab, the student will be able to:

- Explain the Readers-Writers Problem and the need for synchronization.
- Understand the concept of writer priority and how it impacts resource access.
- Apply semaphores (binary and counting) for mutual exclusion and synchronization.
- Design and implement a concurrent program using inter-process/inter-thread communication primitives.

DESCRIPTION

The Readers-Writers Problem involves a shared resource (like a database or file) that can be simultaneously read by multiple reader processes/threads but can only be modified by a single writer process/thread at any given time.

Synchronization Requirements:

1. Multiple Readers: Any number of readers can access the resource concurrently.
2. Exclusive Writer: Only one writer can access the resource at a time.
3. Mutual Exclusion: A writer cannot access the resource while any reader or another writer is accessing it.

Writer Priority:

In the Writer Priority solution, if a writer is waiting, no new reader is allowed to start reading, even if the resource is currently being read by others. New readers must wait until all waiting writers (and any writer currently in the critical section) have finished. This prevents writer starvation.

Semaphores Used:

To implement this, we typically use four semaphores and one shared variable:

Name	Type	Initial value	Purpose
mutex	Binary	1	Ensures mutual exclusion when updating readCount and writeCount (local to readers/writers).
Resource	Binary	1	Ensures mutual exclusion for access to the shared resource itself (only one writer or multiple readers can hold it).
writeBlock	Binary	1	Used by readers/writers to block new readers when a writer is waiting.
writeMutex	Binary	1	Ensures mutual exclusion when updating writeCount.
readCount	Integer	0	Tracks the number of readers currently in the critical section.
writeCount	Integer	0	Tracks the number of writers waiting or currently in the critical section.

ALGORITHM

Step 1: Reader Process

Goal: Allow multiple readers simultaneously, but block if a writer wants to write.

Steps:

1. Try to enter reading section while respecting writer priority:
 - Wait (lock) readTry → ensures no writer is waiting.

- Wait (lock) mutex → protect readcount.
- 2. Increment reader count:
 - readcount++
 - If readcount == 1 → first reader locks wrt (prevents writers from writing).
- 3. Release semaphores:
 - Signal (unlock) mutex.
 - Signal (unlock) readTry → allows others to check writer priority.
- 4. Perform reading:
 - Access shared resources for reading.
- 5. Exit reading section:
 - Wait (lock) mutex.
 - readcount--
 - If readcount == 0 → last reader signals wrt (allow writers to write).
 - Signal (unlock) mutex.

Step 2: Writer Process

Goal: Writers get exclusive access to the resource, with priority over new readers.

Steps:

1. Wait (lock) wrt → ensures exclusive access to shared resources.
2. Perform writing → access shared resources.
3. Signal (unlock) wrt → release resource so next reader or writer can access.

Step 3: Thread Creation

1. Create multiple reader threads.
2. Create multiple writer threads.

Readers and writers run concurrently, but synchronization ensures:

- Multiple readers can read simultaneously.
- Only one writer writes at a time.
- Writers are given priority if they are waiting.

PROGRAM

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
sem_t wrt, mutex, readTry;
int readcount = 0;
void *reader(void *arg) {
    int f = ((int)arg);
    sem_wait(&readTry);
    sem_wait(&mutex);
    readcount++;
    if(readcount == 1)
        sem_wait(&wrt);
    sem_post(&mutex);
    sem_post(&readTry);
    printf("Reader %d is reading\n", f);
    sleep(1);
    sem_wait(&mutex);
    readcount--;
    if(readcount == 0)
        sem_post(&wrt);
    sem_post(&mutex);
}
void *writer(void *arg) {
    int f = ((int)arg);
    sem_wait(&wrt);
    printf("Writer %d is writing\n", f);
    sleep(2);
    sem_post(&wrt);
}
int main() {
    pthread_t r[5], w[5];
```

```
sem_init(&mutex,0,1);
sem_init(&wrt,0,1);
sem_init(&readTry,0,1);
// Create writer threads
for(int i=0;i<5;i++)
    pthread_create(&w[i],NULL,writer,(void *)i);
// Create reader threads
for(int i=0;i<5;i++)
    pthread_create(&r[i],NULL,reader,(void *)i);
for(int i=0;i<5;i++) {
    pthread_join(r[i],NULL);
    pthread_join(w[i],NULL);
}
return 0;
}
```

PROCEDURE

1. Setup: Create a new C/C++ source file (e.g., rw_writer_priority.c).
2. Code Entry: Enter the complete code, including the necessary header files, semaphore initialization, reader function, writer function, and a main function to create and manage the threads/processes.
3. Compilation: Compile the code using the GCC compiler, ensuring to link the pthread library:
`gcc rw_writer_priority.c -o rw_writer_priority -pthread`
4. Execution: Run the compiled program:
`./rw_writer_priority`
5. Observation: Observe the output, noting the sequence of reader and writer access. Introduce delays (e.g., sleep() or usleep()) in the critical sections to make the concurrency effects more visible.
6. Verification: Verify that when a writer is waiting, new readers are blocked, and the writer gets access before any new readers.

EXPECTED OUTPUT

```
Writer 0 is writing
Writer 1 is writing
Reader 0 is reading
Reader 1 is reading
Writer 2 is writing
Reader 2 is reading
```

MODEL VIVA QUESTIONS

1. What is the Readers-Writers Problem?
2. How do you give priority to writers?
3. What are semaphores used for?
4. What is readcount?
5. Can multiple writers write simultaneously?

RESULT

The experiment successfully demonstrated the implementation of the Readers-Writers problem using semaphores, where writers were granted priority access to the shared resource, effectively preventing writer starvation by blocking new readers until all waiting writers completed their tasks.

EXP NO : 11**BANKER'S ALGORITHM SIMULATION****AIM**

The primary objective is to understand and implement the Banker's Algorithm (Safety Algorithm) to determine if a given system state is safe (deadlock-free) and, if so, to find a safe execution sequence of processes.

LEARNING OUTCOMES

Upon completion of this experiment, the student will be able to:

- Define and differentiate between safe and unsafe states in an operating system.
- Understand the purpose and components of the Banker's Algorithm.
- Calculate the Need matrix for a given system state.
- Apply the Safety Algorithm to determine if a system is in a safe state.
- Simulate the process of resource allocation and release to find a safe sequence.

DESCRIPTION**Deadlock and Deadlock Avoidance**

A deadlock occurs when a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process. The Banker's Algorithm is a deadlock avoidance technique, meaning it checks the system state *before* granting a resource request to ensure the allocation won't lead to an unsafe state.

Data Structures

The algorithm relies on the following key data structures, where n is the number of processes and m is the number of resource types:

- Available (Vector, $1 \times m$): The number of available instances of each resource type.
- Max (Matrix, $n \times m$): The maximum demand of each process.

- Allocation (Matrix, $n \times m$): The number of resources of each type currently allocated to each process.
- Need (Matrix, $n \times m$): The remaining resource needed for each process to complete its execution. It is calculated as:
$$\text{Need}[i,j] = \text{Max}[i,j] - \text{Allocation}[i,j]$$

ALGORITHM

Definitions

- n = Number of processes
- m = Number of resource types
- $\text{Available}[m]$ = Number of available resources of each type
- $\text{Max}[n][m]$ = Maximum demand of each process
- $\text{Allocation}[n][m]$ = Current allocation to each process
- $\text{Need}[n][m]$ = Remaining need of each process ($\text{Need} = \text{Max} - \text{Allocation}$)

Banker's Algorithm Steps

Step 1: Initialize $\text{Work} = \text{Available}$ and $\text{Finish}[i] = \text{false}$ for all processes.

Step 2: Find a process i such that:

- $\text{Finish}[i] == \text{false}$
- $\text{Need}[i] \leq \text{Work}$ (for all resources)

Step 3: If such a process is found:

- $\text{Work} = \text{Work} + \text{Allocation}[i]$ (release resources after execution)
- $\text{Finish}[i] = \text{true}$
- Add process i to safe sequence

Step 4: Repeat Steps 2–3 until all processes finish or no such process is found.

Step 5:

- If all $\text{Finish}[i] == \text{true} \rightarrow$ system is safe
- Else $\rightarrow$ system is unsafe, deadlock may occur

PROGRAM

```
#include <stdio.h>

int main() {

    int n, m, i, j, k;

    printf("Enter number of processes: ");

    scanf("%d", &n);

    printf("Enter number of resources: ");

    scanf("%d", &m);

    int Allocation[n][m], Max[n][m], Need[n][m], Available[m];

    int Finish[n], safeSeq[n];

    printf("Enter Allocation Matrix:\n");

    for(i=0;i<n;i++)

        for(j=0;j<m;j++)

            scanf("%d",&Allocation[i][j]);

    printf("Enter Max Matrix:\n");

    for(i=0;i<n;i++)

        for(j=0;j<m;j++)

            scanf("%d",&Max[i][j]);

    printf("Enter Available Resources:\n");

    for(j=0;j<m;j++)

        scanf("%d",&Available[j]);

    // Calculate Need matrix
```

```
for(i=0;i<n;i++)

    for(j=0;j<m;j++)

        Need[i][j] = Max[i][j] - Allocation[i][j];

for(i=0;i<n;i++)

    Finish[i] = 0;

int count = 0;

while(count < n) {

    int found = 0;

    for(i=0;i<n;i++) {

        if(Finish[i]==0) {

            int flag = 1;

            for(j=0;j<m;j++)

                if(Need[i][j] > Available[j])

                    flag = 0;

            if(flag) {

                for(k=0;k<m;k++)

                    Available[k] += Allocation[i][k];

                safeSeq[count++] = i;

                Finish[i] = 1;

                found = 1;

            }

        }

    }

}
```

```

    }

    if(found==0) {

        printf("System is not in safe state.\n");

        return 0;

    }

}

printf("System is in safe state.\nSafe sequence: ");

for(i=0;i<n;i++)

    printf("P%d ", safeSeq[i]);

return 0;

}

```

PROCEDURE

Initialize Data:

- Number of processes PPP and resources RRR.
- Available vector, Max matrix, and Allocation matrix.

Calculate

NeedMatrix:

$Need[i][j] = Max[i][j] - Allocation[i][j]$
 $Need[i][j] = Max[i][j] - Allocation[i][j]$

Check for Safe Sequence:

- Initialize Work = Available and Finish[i] = false for all processes.
- Find a process i such that Finish[i] == false and Need[i] <= Work.
- If found, assume process i executes, then release its allocated resources:
 $Work = Work + Allocation[i]$
- Mark Finish[i] = true and add process i to the safe sequence.
- Repeat until all processes finish or no such process is found.

Determine State:

- If all $\text{Finish}[i] == \text{true}$, the system is safe and the safe sequence is found.
- Otherwise, the system is unsafe.

EXPECTED OUTPUT

Process	Allocation	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available Resources: [3, 3, 2]

Need Matrix:

P0: [7, 4, 3]

P1: [1, 2, 2]

P2: [6, 0, 0]

P3: [0, 1, 1]

P4: [4, 3, 1]

Safe Sequence Found: P1 -> P3 -> P0 -> P2 -> P4

System is in a SAFE state.

MODEL VIVA QUESTIONS

1. What is the Banker's Algorithm?
2. How do you define a safe state?
3. How is the Need matrix calculated?
4. Explain the difference between deadlock avoidance and deadlock prevention.
5. What happens if no safe sequence is found?

RESULT

The Banker's Algorithm simulation successfully demonstrated that the given initial process mix is in a safe state. A safe sequence $\langle P1, P3, P4, P0, P2 \rangle$ was found, guaranteeing that all processes can complete their execution without leading to a deadlock.

EXP NO : 12**DEADLOCK DETECTION****AIM**

To understand the concept of deadlock in a multi-process system, implement an algorithm for its detection (e.g., Banker's Algorithm's Safety Check or a dedicated Deadlock Detection Algorithm), and determine if a given system state is safe or deadlocked.

LEARNING OUTCOMES

Upon completion of this lab, the student will be able to:

1. Define the four necessary conditions for a deadlock to occur (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait).
2. Analyze a given system state (Available, Max, Allocation matrices) for resource management.
3. Implement and execute the Deadlock Detection Algorithm (or Banker's Safety Algorithm).
4. Determine if a system is in a safe state (no deadlock) or a deadlocked state, and identify the processes involved in the deadlock.

DESCRIPTION

A deadlock is a situation where two or more processes are blocked indefinitely, waiting for a resource held by another blocked process.

The Banker's Algorithm is primarily a deadlock avoidance algorithm, but its core logic (the safety check) is often used for detection. It operates by checking if a system is in a safe state.

System State Representation

The system state is defined by the following matrices:

- n : Number of Processes ($P_0, P_1, \dots, P_{n-1}$)
- m : Number of Resource Types ($R_0, R_1, \dots, R_{m-1}$)

1. Allocation ($n \times m$ matrix): Allocation[i,j] is the number of instances of resource R_j currently allocated to process P_i .
2. Max ($n \times m$ matrix): Max[i,j] is the maximum number of instances of resource R_j that process P_i may request.
3. Available ($1 \times m$ vector): Available[j] is the number of available instances of resource R_j .
4. Need ($n \times m$ matrix): Need[i,j] is the remaining resource demand for process P_i .
Need=Max–Allocation

Deadlock Detection (Safety) Algorithm

1. Initialize Work vector (size m) = Available vector.
2. Initialize Finish vector (size n) = false for all processes.
3. Find an index i such that:
 - Finish[i]==false
 - Need $i \leq$ Work (Process P_i 's needs can be met by the current available resources).
4. If such an i exists, simulate the execution and release of P_i 's resources:
 - Work=Work+Allocation $_i$
 - Finish[i]=true
 - Go to step 3.
5. If no such i can be found, the loop terminates.
6. Safety Check: If Finish[i]==true for all i, the system is in a safe state (no deadlock). Otherwise, the processes for which Finish[i] is false are deadlocked.

ALGORITHM

Initialize:

- Work = Available
- Finish[i] = false if Allocation[i] $\neq$ 0; true if Allocation[i] = 0

Find a process $P[i]$ such that:

- Finish[i] == false
- Need[i] $\leq$ Work

If such a process exists:

- Work = Work + Allocation[i]

- Finish[i] = true
- Repeat step 2

If no such process exists:

- Processes with Finish[i] == false are deadlocked

PROGRAM

```
#include <stdio.h>
#include <stdbool.h>
#define P 5 // Number of processes
#define R 3 // Number of resources
int main() {
    int i, j;
    // Example Data
    int Available[R] = {0, 0, 0}; // Available instances of each resource
    int Allocation[P][R] = { // Allocation matrix
        {0, 1, 0}, // P0
        {2, 0, 0}, // P1
        {3, 0, 2}, // P2
        {2, 1, 1}, // P3
        {0, 0, 2} // P4
    };
    int Max[P][R] = { // Maximum demand matrix
        {7, 5, 3}, // P0
        {3, 2, 2}, // P1
        {9, 0, 2}, // P2
        {2, 2, 2}, // P3
        {4, 3, 3} // P4
    };
    int Need[P][R]; // Need matrix
    bool Finish[P] = {false}; // Finish array
    int Work[R]; // Work vector
    // Step 1: Calculate Need matrix
```

```
for(i = 0; i < P; i++) {
    for(j = 0; j < R; j++) {
        Need[i][j] = Max[i][j] - Allocation[i][j];
    }
}

// Step 2: Initialize Work = Available
for(j = 0; j < R; j++) {
    Work[j] = Available[j];
}

// Step 3: Deadlock Detection Algorithm
bool executed;
do {
    executed = false;
    for(i = 0; i < P; i++) {
        if(!Finish[i]) {
            bool canExecute = true;
            for(j = 0; j < R; j++) {
                if(Need[i][j] > Work[j]) {
                    canExecute = false;
                    break;
                }
            }
            if(canExecute) {
                for(j = 0; j < R; j++)
                    Work[j] += Allocation[i][j];
                Finish[i] = true;
                executed = true;
            }
        }
    }
} while(executed);

// Step 4: Print Need matrix
printf("Available Resources: ");
for(j = 0; j < R; j++)
```

```

    printf("%d ", Available[j]);
printf("\n");
printf("Need Matrix:\n");
for(i = 0; i < P; i++) {
    printf("P%d: ", i);
    for(j = 0; j < R; j++)
        printf("%d ", Need[i][j]);
    printf("\n");
}
// Step 5: Identify deadlocked processes
bool deadlock = false;
printf("\n");
printf("Deadlocked Processes: ");
for(i = 0; i < P; i++) {
    if(!Finish[i]) {
        printf("P%d ", i);
        deadlock = true;
    }
}
if(deadlock)
    printf("\nDeadlock Detected!\n");
else
    printf("\nNo Deadlock. All processes can finish safely.\n");
return 0;
}

```

PROCEDURE

1. Obtain the number of processes and resource
2. Define the Available vector, Max matrix, and Allocation matrix.
Compute Need matrix: $Need[i][j] = Max[i][j] - Allocation[i][j]$
3. Apply the Deadlock Detection Algorithm.

4. If all Finish[i] = true → No deadlock.
 Else → Deadlock exists; list deadlocked processes.

EXPECTED OUTPUT

Process	Allocation	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Available Resources: [0, 0, 0]

Need Matrix:

P0: [7, 4, 3]

P1: [1, 2, 2]

P2: [6, 0, 0]

P3: [0, 1, 1]

P4: [4, 3, 1]

Deadlock Detected!

Deadlocked Processes: P0, P1, P2, P3, P4

MODEL VIVA QUESTIONS

1. What is a deadlock?
2. List the necessary conditions for deadlock.
3. Difference between deadlock prevention, avoidance, and detection.
4. How is the Need matrix calculated?
5. Explain the Deadlock Detection Algorithm.

RESULT

The program executed the Banker's Safety Algorithm on the given process mix. The final result is that the system is in a safe state, and therefore not deadlocked, as a safe sequence, ⟨P1,P3,P0,P2,P4⟩, was found.

EXP NO : 13

DEADLOCK-FREE DINING PHILOSOPHERS SOLUTION USING SEMAPHORES

AIM

The primary objective of this experiment is to implement and test a semaphore-based solution for the classic Dining Philosophers Problem that prevents deadlock. This demonstrates fundamental principles of process synchronization and resource management in an operating system.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

- Understand the classic Deadlock problem and the specific conditions (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait) that cause it in the Dining Philosophers scenario.
- Apply semaphores for implementing mutual exclusion and process synchronization.
- Implement a safe resource allocation strategy (limiting simultaneous philosophers) to guarantee a deadlock-free system.
- Analyze the behavior of concurrent processes and their interaction with shared resources.

DESCRIPTION

The Dining Philosophers Problem

The Dining Philosophers Problem is a classic concurrency problem introduced by E.W. Dijkstra. N philosophers are seated around a circular table, each with a bowl of spaghetti. Between each pair of adjacent philosophers is a single fork. A philosopher must use both the left and right forks to eat. When they are not eating, they are thinking.

The Deadlock Problem: The naive solution is for each philosopher to pick up their left fork, and then their right fork. If all N philosophers simultaneously pick up their left fork, they will all be waiting indefinitely for the right fork, which is held by their neighbor. This is a state of deadlock.

Deadlock-Free Strategy (Limited Concurrency)

To prevent the Circular Wait condition, which is essential for deadlock, we introduce a restriction: a maximum of $N-1$ philosophers can sit at the table (or, more precisely, be in the critical section of trying to pick up forks) at any given time.

This restriction is enforced by a single counting semaphore, often called the 'room' or 'butler' semaphore, initialized to $N-1$.

- Philosopher (Process): Before attempting to pick up the forks, the philosopher must *acquire* the room semaphore (a `wait()` operation).
- Room Semaphore: If the room is full (i.e., $N-1$ other philosophers are trying to eat), the current philosopher blocks.
- When Done: After eating and putting down the forks, the philosopher *releases* the room semaphore (a `signal()` operation).

This ensures that at least one philosopher will always be able to pick up both forks, preventing the circular waiting chain.

ALGORITHM

1. Initialize semaphores:

 `mutex = 1` (binary semaphore)

 `fork[i] = 1` for each fork
2. For philosopher i :

 do {

 `think();`

 `wait(mutex);` // enter critical section

 `wait(fork[min(i, (i+1)%N]));`

 `wait(fork[max(i, (i+1)%N]));`

 `signal(mutex);` // leave critical section

 `eat();`

```
        signal(fork[i]);    // put down forks
```

```
        signal(fork[(i+1)%N]);
```

```
    } while(true);
```

3. Repeat for all philosophers.

PROGRAM

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#define N 5 // Number of philosophers
```

```
sem_t forks[N];
```

```
sem_t mutex;
```

```
void think(int i) {
```

```
    printf("Philosopher %d is thinking.\n", i);
```

```
    sleep(1); // Simulate thinking
```

```
}
```

```
void eat(int i) {
```

```
    printf("Philosopher %d is eating.\n", i);
```

```
    sleep(1); // Simulate eating
```

```
}
```

```
void* philosopher(void* num) {
```

```
    int i = *(int*)num;
```

```
    int left = i;
```

```
    int right = (i + 1) % N;
```

```
    while(1) {
```

```
        think(i);
```

```
        sem_wait(&mutex); // enter critical section
```

```
        // Pick up forks in order to avoid deadlock
```

```
        if(left < right) {
```

```
            sem_wait(&forks[left]);
```

```
            sem_wait(&forks[right]);
```

```
    } else {
        sem_wait(&forks[right]);
        sem_wait(&forks[left]);
    }
    sem_post(&mutex); // leave critical section
    eat(i);
    // Put down forks
    sem_post(&forks[left]);
    sem_post(&forks[right]);
}
return NULL;
}

int main() {
    pthread_t phil[N];
    int i;
    sem_init(&mutex, 0, 1);
    for(i = 0; i < N; i++)
        sem_init(&forks[i], 0, 1);
    int phil_num[N];
    for(i = 0; i < N; i++) {
        phil_num[i] = i;
        pthread_create(&phil[i], NULL, philosopher, &phil_num[i]);
    }
    for(i = 0; i < N; i++)
        pthread_join(phil[i], NULL);
    for(i = 0; i < N; i++)
        sem_destroy(&forks[i]);
    sem_destroy(&mutex);
    return 0;
}
```

PROCEDURE

Initialize fork semaphores and mutex.

Each philosopher performs the following in a loop:

- Think for a random time.
- Wait for the two forks (using semaphores).
- Eat for a random time.
- Release the forks (signal the semaphores).

Repeat steps indefinitely or for a fixed number of iterations.

Ensure deadlock-free operation by acquiring forks in numerical order.

EXPECTED OUTPUT

```
Philosopher 0 is thinking.  
Philosopher 1 is thinking.  
Philosopher 2 is thinking.  
Philosopher 0 is eating.  
Philosopher 3 is thinking.  
Philosopher 1 is eating.  
Philosopher 4 is thinking.  
Philosopher 2 is eating.  
...
```

MODEL VIVA QUESTIONS

1. What is the Dining Philosophers Problem?
2. What causes deadlock in this problem?
3. How does the deadlock-free solution prevent circular wait?
4. Explain the role of mutex and fork semaphores.
5. Can this solution be extended to N philosophers?

RESULT

The deadlock-free solution for the Dining Philosophers Problem was successfully implemented using POSIX semaphores. The use of a counting semaphore (the room semaphore) initialized to $N-1$ effectively limited the number of philosophers concurrently attempting to acquire resources.

EXP NO : 14**PAGING ADDRESS TRANSLATION SIMULATION****AIM**

To understand and simulate the mechanism of virtual-to-physical address translation in a computer system using the paging memory management scheme.

LEARNING OUTCOMES

Upon successful completion of this lab, the student will be able to:

- Calculate the number of bits required for the Virtual Address (VA) and Physical Address (PA).
- Determine the page size and frame size in terms of bytes and the number of bits for the page/frame offset.
- Decompose a Virtual Address into a Virtual Page Number (VPN) and an Offset.
- Simulate a Page Table lookup to find the corresponding Physical Frame Number (PFN).
- Construct the Physical Address from the PFN and Offset.
- Identify a Page Table Miss scenario.

DESCRIPTION

Paging is a memory management scheme that eliminates external fragmentation and allows a program's physical address space to be non-contiguous.

1. Virtual Address (VA): The address generated by the CPU. It is divided into two parts:
 - Virtual Page Number (VPN): Used as an index into the Page Table.
 - Offset: The displacement within the page/frame. $VA = \langle VPN, Offset \rangle$
2. Page Table: A per-process data structure that maps VPNs to Physical Frame Numbers (PFNs).
3. Physical Address (PA): The actual address in physical memory. It is constructed from the PFN and the Offset. $PA = \langle PFN, Offset \rangle$

Calculation Formulas:

- Number of bits for Offset (Obits):
 $\text{Obits} = \log_2(\text{Page Size in bytes})$
- Number of bits for VPN (VPNbits):
 $\text{VPNbits} = \log_2(\text{Page Size in bytes} \times \text{Virtual Address Space Size in bytes})$
- Total VA bits (VAbits):
 $\text{VAbits} = \text{VPNbits} + \text{Obits}$

ALGORITHM

1. **Input:** Virtual address space size (MB), page size (KB), virtual address (decimal).

2. **Convert sizes:**

- Virtual address space $\rightarrow$ bytes
- Page size $\rightarrow$ bytes

3. **Calculate page number and offset:**

$\text{Page_number} = \text{virtual_address} / \text{page_size_in_bytes}$

$\text{Offset} = \text{virtual_address} \% \text{page_size_in_bytes}$

4. **Access page table:**

- If page_number exists $\rightarrow$ $\text{frame_number} = \text{page_table}[\text{page_number}]$
- Else $\rightarrow$ declare **page table miss**

5. **Output:** <frame number, offset>

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    if(argc != 4) {
        printf("Usage: %s <virtual_space_MB> <page_size_KB> <virtual_address>\n", argv[0]);
        return 1;
    }
    int virtual_space_MB = atoi(argv[1]);
    int page_size_KB = atoi(argv[2]);
```

```
unsigned int virtual_address = atoi(argv[3]);
// Convert sizes to bytes
unsigned int virtual_space_bytes = virtual_space_MB * 1024 * 1024;
unsigned int page_size_bytes = page_size_KB * 1024;
// Calculate number of pages
int num_pages = virtual_space_bytes / page_size_bytes;
// Simulate page table (frame numbers)
int page_table[num_pages];
for(int i = 0; i < num_pages; i++) {
    page_table[i] = i + 10; // Just example frame numbers starting from 10
}
// Calculate page number and offset
int page_number = virtual_address / page_size_bytes;
int offset = virtual_address % page_size_bytes;
if(page_number >= num_pages) {
    printf("Page Table Miss! Invalid page number.\n");
} else {
    int frame_number = page_table[page_number];
    printf("Virtual Address %u -> Physical Address: <%d, %d>\n", virtual_address, frame_number,
offset);
}
return 0;
}
```

PROCEDURE

1. Initialize a page table as an array.
2. Read the command-line arguments:
 - a. Virtual address space (MB)
 - b. Page size (KB)
 - c. Virtual address (decimal)
3. Calculate page number and offset.
4. Lookup page table for the frame number.
5. Print <frame number, offset> or page table miss.

EXPECTED OUTPUT

```
Page Number = 7000 / 4096 = 1  
Offset = 7000 % 4096 = 2904  
Frame Number = page_table[1] = 11 (as per our example table)
```

```
Virtual Address 7000 -> Physical Address: <11, 2904>
```

MODEL VIVA QUESTIONS

1. What is paging and why is it used?
2. How is a virtual address divided in paging?
3. Define page number and offset.
4. What happens in case of a page table miss?
5. How is physical address calculated from virtual address?

RESULT

The program successfully translates a virtual address to a physical address using the paging scheme. If the page number is invalid or not in the page table → Page Table Miss is reported.

EXP NO : 15

SIMULATION OF PAGE REPLACEMENT ALGORITHMS

AIM

To understand and implement the First-In, First-Out (FIFO), Least Recently Used (LRU), and Optimal (OPT) page replacement algorithms and compare their performance in terms of the number of page faults for a given page-reference string and a varying number of available page frames.

LEARNING OUTCOMES

Upon completion of this lab, the student will be able to:

1. Generate a random page-reference string for simulation.
2. Implement the logic for the FIFO, LRU, and Optimal page replacement algorithms.
3. Analyze the performance of different page replacement algorithms.
4. Differentiate between the algorithms based on their page fault count.
5. Explain the concept of demand paging and page faults.

DESCRIPTION

In a system using demand paging, a page is only loaded into physical memory (a frame) when it is referenced. If a reference is made to a page not currently in memory, a page fault occurs. If all available page frames are full, a page replacement algorithm is required to select a page currently in memory to be swapped out to make room for the newly referenced page.

- FIFO: The page that has been in memory for the longest time is replaced.
- LRU: The page that has not been used for the longest period of time is replaced.
- Optimal: The page that will not be used for the longest period of time in the future is replaced.

This is often called the perfect algorithm as it gives the minimum possible page faults but is unimplementable in a real OS.

ALGORITHM

1. FIFO

1. Maintain a queue of pages in memory.
2. If page fault occurs and memory is full → remove the first page in the queue.
3. Insert the new page at the end of the queue.

2. LRU

1. Track last used time of each page in memory.
2. On page fault, remove the page with the earliest last used time.

3. Optimal

1. Look ahead in the reference string to determine future use.
2. Replace the page that will not be used for the longest time.

PROGRAM

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#define MAX_PAGE 10

// Function to generate random page reference string

void generate_reference_string(int ref[], int length) {

    for(int i=0; i<length; i++) {

        ref[i] = rand() % MAX_PAGE;

    }

}
```

```
// FIFO Page Replacement
```

```
int fifo(int ref[], int n, int frames) {  
  
    int page[frames];  
  
    int front = 0, count = 0, page_faults = 0;  
  
    for(int i=0; i<frames; i++) page[i] = -1;  
  
    for(int i=0; i<n; i++) {  
  
        int found = 0;  
  
        for(int j=0; j<frames; j++) {  
  
            if(page[j] == ref[i]) { found = 1; break; }  
  
        }  
  
        if(!found) {  
  
            page[front] = ref[i];  
  
            front = (front + 1) % frames;  
  
            page_faults++;  
  
        }  
  
    }  
  
    return page_faults;  
}
```

```
// LRU Page Replacement
```

```
int lru(int ref[], int n, int frames) {  
  
    int page[frames];  
  
    int recent[frames];
```

```
int page_faults = 0;

for(int i=0; i<frames; i++) { page[i] = -1; recent[i] = -1; }

for(int i=0; i<n; i++) {

    int found = -1;

    for(int j=0; j<frames; j++) {

        if(page[j] == ref[i]) { found = j; break; }

    }

    if(found != -1) { // Update recent

        recent[found] = i;

    } else { // Page fault

        int replace = 0;

        for(int j=1; j<frames; j++) {

            if(recent[j] < recent[replace]) replace = j;

        }

        page[replace] = ref[i];

        recent[replace] = i;

        page_faults++;

    }

}

return page_faults;

}
```

// Optimal Page Replacement

```
int optimal(int ref[], int n, int frames) {  
  
    int page[frames];  
  
    int page_faults = 0;  
  
    for(int i=0; i<frames; i++) page[i] = -1;  
  
    for(int i=0; i<n; i++) {  
  
        int found = 0;  
  
        for(int j=0; j<frames; j++)  
  
            if(page[j] == ref[i]) { found = 1; break; }  
  
        if(!found) {  
  
            int replace = -1, farthest = i;  
  
            for(int j=0; j<frames; j++) {  
  
                int k;  
  
                for(k=i+1; k<n; k++) if(page[j] == ref[k]) break;  
  
                if(k > farthest) { farthest = k; replace = j; }  
  
            }  
  
            if(replace == -1) replace = 0;  
  
            page[replace] = ref[i];  
  
            page_faults++;  
  
        }  
  
    }  
  
    return page_faults;  
  
}
```

```
int main(int argc, char *argv[]) {

    if(argc != 3) {

        printf("Usage: %s <reference_string_length> <max_frames>\n", argv[0]);

        return 1;

    }

    int n = atoi(argv[1]);

    int max_frames = atoi(argv[2]);

    int ref[n];

    srand(time(NULL));

    generate_reference_string(ref, n);

    printf("Random Page Reference String: ");

    for(int i=0; i<n; i++) printf("%d ", ref[i]);

    printf("\n\n");

    printf("Frames\tFIFO\tLRU\tOptimal\n");

    for(int f=1; f<=max_frames; f++) {

        int fifo_faults = fifo(ref, n, f);

        int lru_faults = lru(ref, n, f);

        int opt_faults = optimal(ref, n, f);

        printf("%d\t%d\t%d\t%d\n", f, fifo_faults, lru_faults, opt_faults);

    }

    return 0;

}
```

PROCEDURE

1. Read command-line arguments:
 - a. Length of page reference string
 - b. Maximum number of page frames
2. Generate a **random page reference string** (page numbers 0–9).
3. For **frame sizes from 1 to 7**: Apply **FIFO, LRU, and Optimal algorithms**.
Count the number of **page faults** for each algorithm
4. Display the **page reference string, frame size, and page faults**

EXPECTED OUTPUT

Random Page Reference String: 1 3 0 4 1 2 1 3 0 2

Frames	FIFO	LRU	Optimal
1	10	10	10
2	8	7	6
3	6	5	5
4	5	4	4
5	4	4	3
6	4	3	3
7	3	3	2

MODEL VIVA QUESTIONS

1. Which algorithm gives minimum page faults? Why?
2. What is Belady's Anomaly?
3. How does frame size affect page faults?
4. How is LRU implemented efficiently?
5. Difference between demand paging and pre-paging.

RESULT

The experiment successfully simulated the FIFO, LRU, and Optimal page replacement algorithms. The results show that the Optimal algorithm consistently produces the minimum number of page faults, confirming its theoretical efficiency. The LRU algorithm performs better than FIFO because it replaces the page that is least likely to be used in the near future (by replacing the one not used in the past).

EXP NO : 16**SIMULATION OF DISK SCHEDULING ALGORITHMS (SSTF, LOOK, C-SCAN)****AIM**

The primary objective is to simulate and compare the performance of the Shortest Seek Time First (SSTF), LOOK, and Circular SCAN (C-SCAN) disk-scheduling algorithms by calculating the total number of head movements required to service a random sequence of cylinder requests.

LEARNING OUTCOMES

Upon successful completion of this experiment, the student will be able to:

- Understand the fundamental concepts of disk scheduling and the need for optimization.
- Explain the working principles of SSTF, LOOK, and C-SCAN algorithms.
- Implement the logic for these algorithms using a programming language.
- Analyze the performance (total head movement) of different scheduling algorithms for a given set of requests.

DESCRIPTION

Disk scheduling is a process that organizes the order in which pending I/O requests are serviced by the disk head. The goal is to minimize the total seek time, which is directly proportional to the total distance (number of cylinders) the disk head moves. The disk has 5,000 cylinders, numbered from 0 to 4,999.

- Shortest Seek Time First (SSTF): This algorithm selects the request that requires the minimum head movement from its current position. It provides a good performance but can lead to starvation for requests far from the head.
- LOOK: An improved version of SCAN. The head moves in one direction (e.g., 0 to 4999), servicing all requests in its path. When there are no more requests in the current direction, it immediately reverses direction (instead of going all the way to the end of the disk like pure SCAN).
- Circular SCAN (C-SCAN): The head moves only in one direction (e.g., 0 to 4999). When it reaches the end of the disk, it immediately jumps back to the beginning (cylinder 0) without servicing any requests during the jump, and then begins its sweep again. This provides a more uniform wait time compared to SCAN/LOOK.

ALGORITHM**1. SSTF**

1. Start from the **initial head position**.
2. Find the request **closest** to the current head.
3. Move the head to that request and mark it as serviced.
4. Repeat until all requests are serviced.
5. Sum all head movements.

2. LOOK

1. Sort requests in ascending order.
2. Move the head in **increasing order**, servicing requests greater than or equal to initial position.
3. Reverse direction at the furthest request and service remaining requests in decreasing order.
4. Sum all head movements.

3. C-SCAN

1. Sort requests in ascending order.
2. Move the head in **increasing order**.
3. After the furthest request, move the head to the **beginning of the disk (0)** and continue servicing.
4. Sum all head movements.

PROGRAM

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <time.h>

#define DISK_SIZE 5000

#define NUM_REQUESTS 10

void generate_requests(int requests[]) {

    for(int i=0; i<NUM_REQUESTS; i++)
```

```
    requests[i] = rand() % DISK_SIZE;
}

// SSTF Disk Scheduling

int sstf(int requests[], int n, int head) {

    int serviced[n];

    for(int i=0;i<n;i++) serviced[i] = 0;

    int total_movement = 0, current = head;

    for(int i=0;i<n;i++){

        int min_dist = DISK_SIZE, index = -1;

        for(int j=0;j<n;j++){

            if(!serviced[j] && abs(current - requests[j]) < min_dist){

                min_dist = abs(current - requests[j]);

                index = j;

            }

        }

        total_movement += min_dist;

        current = requests[index];

        serviced[index] = 1;

    }

    return total_movement;

}

// LOOK Disk Scheduling
```

```
int look(int requests[], int n, int head) {

    int sorted[n];

    for(int i=0;i<n;i++) sorted[i] = requests[i];

    // Sort requests

    for(int i=0;i<n-1;i++)

        for(int j=i+1;j<n;j++)

            if(sorted[i] > sorted[j]) {

                int temp = sorted[i];

                sorted[i] = sorted[j];

                sorted[j] = temp;

            }

    int total_movement = 0;

    int current = head;

    // Move in increasing direction first

    int i;

    for(i=0;i<n;i++) if(sorted[i] >= current) break;

    for(int j=i;j<n;j++) {

        total_movement += abs(current - sorted[j]);

        current = sorted[j];

    }

    // Move in decreasing direction

    for(int j=i-1;j>=0;j--) {
```

```
        total_movement += abs(current - sorted[j]);

        current = sorted[j];
    }

    return total_movement;
}

// C-SCAN Disk Scheduling

int cscan(int requests[], int n, int head) {

    int sorted[n];

    for(int i=0;i<n;i++) sorted[i] = requests[i];

    // Sort requests

    for(int i=0;i<n-1;i++)

        for(int j=i+1;j<n;j++)

            if(sorted[i] > sorted[j]) {

                int temp = sorted[i];

                sorted[i] = sorted[j];

                sorted[j] = temp;

            }

    int total_movement = 0;

    int current = head;

    int i;

    for(i=0;i<n;i++) if(sorted[i] >= current) break;

    // Move in increasing order
```

```
for(int j=i;j<n;j++) {

    total_movement += abs(current - sorted[j]);

    current = sorted[j];

}

// Move from last request to start

if(i>0) {

    total_movement += abs(current - 0);

    current = 0;

    for(int j=0;j<i;j++){

        total_movement += abs(current - sorted[j]);

        current = sorted[j];

    }

}

return total_movement;

}

int main(int argc, char *argv[]) {

    if(argc != 2) {

        printf("Usage: %s <initial_head_position>\n", argv[0]);

        return 1;

    }

    int head = atoi(argv[1]);

    if(head <0 || head >= DISK_SIZE) {
```

```
    printf("Invalid head position. Must be 0-4999.\n");

    return 1;

}

srand(time(NULL));

int requests[NUM_REQUESTS];

generate_requests(requests);

printf("Random Disk Requests: ");

for(int i=0;i<NUM_REQUESTS;i++) printf("%d ", requests[i]);

printf("\nInitial Head Position: %d\n\n", head);

int sstf_moves = sstf(requests, NUM_REQUESTS, head);

int look_moves = look(requests, NUM_REQUESTS, head);

int cscan_moves = cscan(requests, NUM_REQUESTS, head);

printf("Algorithm\tTotal Head Movement\n");

printf("SSTF\t\t%d\n", sstf_moves);

printf("LOOK\t\t%d\n", look_moves);

printf("C-SCAN\t\t%d\n", cscan_moves);

return 0;

}
```

PROCEDURE

1. Setup: Create a new program file (e.g., disk_scheduler.py or disk_scheduler.c).
2. Implementation: Implement the Request Generation function to generate a random list of 10 requests within the 0-4999 range.
3. Implement the three simulation functions (SSTF, LOOK, C-SCAN) based on their respective algorithms.

4. Compilation/Execution: Compile (if C/C++) or execute the program, passing the initial head position as a command-line argument.

- *Example execution:* ./disk_scheduler 2500 (Sets initial head at cylinder 2500)
- **Observation:** Record the total head movements and the service order for all three algorithms.
- **Analysis:** Compare the results and draw conclusions.

EXPECTED OUTPUT

```
Random Disk Requests: 1432 2999 0 2500 4000 1200 3500 499 1000 4500
Initial Head Position: 2000
```

Algorithm	Total Head Movement
SSTF	6676
LOOK	7200
C-SCAN	10998

MODEL VIVA QUESTIONS

1. What is disk scheduling? Why is it needed?
2. Explain SSTF, LOOK, and C-SCAN algorithms.
3. Which algorithm minimizes total head movement?
4. What is starvation in disk scheduling?
5. How does C-SCAN ensure fairness?

RESULT

The program successfully simulates SSTF, LOOK, and C-SCAN disk scheduling algorithms. It reports the total head movements required for each algorithm. SSTF generally has the least total head movement, but may cause starvation. LOOK is more uniform and prevents starvation. C-SCAN ensures fairness, treating the disk as a circular list.